

University of Erlangen-Nuremberg

Multimedia Communications and Signal Processing

Prof. Dr.-Ing. Walter Kellermann

Bachelor Thesis

**Automatic speech recognition with the
Kaldi Toolkit**

Julian David Fritsch

June 2014

Professor: Prof. Dr.-Ing. Walter Kellermann

Supervisors: Prof. Dr. Dr. Ulrich Hoppe

M.Sc. Christian Hümmer

Bachelor thesis

for

Mr. cand. Ing. Julian Fritsch

Automatic speech recognition with the Kaldi Toolkit

The task of automatic speech recognition (ASR) can be described as the estimation of a word sequence given a recorded speech signal. For this purpose, the relevant information in the time-frequency pattern of the speech signal is extracted to obtain a feature representation with minimum dimensionality. These features are supposed to contain all necessary information to classify phonemes defined as the smallest distinctive speech units. The estimation process is based on an acoustical and a language model whose parameters are determined in a training phase with known utterances. This training data generally consists of undistorted speech signals which results in a mismatch between training and deployment environments for distant-talking speech recognition: The reverberation caused by a large speaker-to-microphone distance degrades the recognition performance and makes medical applications, like hands-free interfaces for computer interaction, very challenging.

A very common open source software for speech recognition research is the Hidden Markov Model Toolkit (HTK). However, this toolkit shows disadvantages with respect to flexibility and math support. The main goal of this thesis is to apply the open-source Kaldi software for the task of ASR. The student is asked to set up a speech recognition toolkit for the Wall Street Journal (WSJ) corpus of November 1992 by using the "trunk" version of the Kaldi Toolkit. This includes the use of predefined functions for the data preparation as well as for the training and testing with the following baseline: The student is asked to integrate the language model and grammar provided by the WSJ corpus. Furthermore, the acoustic model shall be based on Hidden Markov models with emission probability density functions based on Gaussian mixture models. The recognition performance (word error rate) and the computational time are both to be compared between the HTK and Kaldi Toolkit for the data provided by the WSJ corpus. In addition to this, the student is asked to convolve the clean speech signals of the test data with different measured room impulse responses and to evaluate the performance dependent on the reverberation time of the acoustic environments. Finally, the recognition performance shall be evaluated for this reverberated test data by applying the parameter adaptation scheme of maximum likelihood linear regression.

Begin date: 15.01.2014


(Prof. Dr.-Ing. Kellermann)

Erklärung

Ich versichere, dass ich die vorliegende Arbeit ohne fremde Hilfe und ohne Benutzung anderer als der angegebenen Quellen angefertigt habe, und dass die Arbeit in gleicher oder ähnlicher Form noch keiner anderen Prüfungsbehörde vorgelegen hat und von dieser als Teil einer Prüfungsleistung angenommen wurde. Alle Ausführungen, die wörtlich oder sinngemäß übernommen wurden, sind als solche gekennzeichnet.

Ort, Datum

Unterschrift

Contents

Abstract	III
Abbreviations and Acronyms	V
1 Introduction	1
2 Fundamentals of Speech Recognition	3
2.1 Speech Corpus	3
2.2 Feature Extraction and Preprocessing	4
2.2.1 Extration of MFCC-Features	4
2.2.2 Feature-space Transformations	6
2.3 Acoustic Model and Viterbi Training	7
2.4 Viterbi Decoding	10
2.5 Challenges in Distant-Talking Speech Recognition	12
3 Kaldi Toolkit	15
3.1 General Overview of Kaldi	15
3.2 Speech Recognition with Weighted Finite-State Transducers	16
3.3 Graph Creation in Kaldi	20
4 Speech Recognition System	22
4.1 Speech Recognition Pipeline	22
4.2 Creation of Reverberated Test Data	33

5	Experimental Results	35
5.1	Performance Comparison between Kaldi and HTK	35
5.2	Different Pruning Beam-Widths	36
5.3	Reverberated Test Data	37
5.4	Feature-space Transformation	37
6	Conclusions	39
	List of Figures	40
	List of Tables	42
	Bibliography	44

Abstract

In this thesis, we present the Kaldi Toolkit, which is an open-source software for scientific research on automatic speech recognition. The essential functionalities of the Kaldi Toolkit are explained by setting up a speech recognition system which can be split into the following parts:

First, the clean speech data and different language models are extracted from the speech corpus (the Wall Street Journal corpus of November 1992). Subsequent to this, MFCC-features are extracted from the test and training data. In the training phase, the parameters of the acoustic model (consisting of HMMs and GMMs) are estimated based on training data with known transcriptions between the utterances. In the recognition phase, a decoding graph is created by composing the acoustic model with a context-dependency model, a lexicon and a language model in order to estimate a word sequence given some speech data. To evaluate the performance of Kaldi, we consider the recognition accuracy (in terms of the word error rate) for clean speech data compared to the HTK Toolkit. Experimental results are also presented in different reverberant environments, and with different feature-space transformations which are part of the standard recipe of the Kaldi Toolkit.

Abbreviations and Acronyms

ASR	Automatic Speech Recognition
LDA	Linear Discriminant Analysis
MLLT	Maximum Likelihood Linear Transformation
SAT	Speaker Adaptive Training
fMLLR	feature-space Maximum Likelihood Linear Regression
WFST	Weighted Finite-State Transducer
HTK	Hidden Markov Model Toolkit
WSJ	Wall Street Journal
LVCSR	Large-Vocabulary Continuous Speech Recognition
SD	Speaker-Dependent
SI	Speaker-Independent
MFCC	Mel-Frequency Cepstral Coefficient
DFT	Discrete Fourier Transform
DCT	Discrete Cosine Transformation
MLLR	Maximum Likelihood Linear Regression
HMM	Hidden Markov Model
FVS	Feature Vector Sequence
GMM	Gaussian Mixture Model
RIR	Room Impulse Response
ID	Identifier
CMVN	Cepstral Mean and Variance Normalization
WER	Word Error Rate

1 Introduction

Automatic Speech Recognition (ASR) has been a popular subject since decades due to the increasing demand for intuitive human-machine interfaces. This concerns not only smartphones, but also controlling devices in smart living systems like interactive TV or voice-activated appliances for disabled persons [1]. Advances in this field of research have already resulted in famous systems like Siri [2]. But striving for speech recognition performance at human-level would not only advance consumer industry, but also medical applications. Medical documentation, as well as hands-free controlling devices, like surgical robotics, imaging devices, air-conditioning or lighting systems could accelerate processes in hospitals and contribute to more sterility [3]. Though, ASR systems have to fulfill certain standards, especially in terms of reliability, as they are used under challenging conditions. These include inter-speaker variability, several different noise sources, but also to overcome the distance between speaker and microphone [4]. The latter involves interfering reflections from walls and objects in the acoustic environment (known as reverberation), which distort the captured signal [4]. This implies, that if an ASR system shall perform well under these conditions, it has to annihilate as much variability as possible [5].

This thesis gives an overview of the functionality and the ambit of the open-source Kaldi Toolkit. Thereby, the performance and computational time is investigated. Further, the impact of reverberation on the recognition accuracy is examined for different feature-space transformations which are part of the standard Kaldi recipe, like a combination of Linear Discriminant Analysis (LDA) and Maximum Likelihood Linear

Transformation (MLLT) or a concatenation of Speaker Adaptive Training (SAT) and feature-space Maximum Likelihood Linear Regression (fMLLR). The thesis is organized as follows: Chapter 2 introduces fundamentals of speech recognition, including feature extraction and preprocessing, acoustic model training, Viterbi decoding and challenges in distant-talking speech recognition. Next, Chapter 3 presents the Kaldi project itself, the principle of Weighted Finite-State Transducers (WFST) and their application for graph creation in Kaldi. The setup and the functionality of the speech recognition pipeline, as well as the creation of reverberated test data are viewed in Chapter 4, followed by a presentation of the experimental results in Chapter 5. This includes a comparison to the Hidden Markov Model Toolkit (HTK) for clean speech data, an investigation of the impact of the pruning beam-width, recognition results for reverberated test data and the performance enhancement gained by applying the feature-space transformations. Finally, Chapter 6 draws conclusions and describes possible future work.

2 Fundamentals of Speech Recognition

This chapter reviews the fundamentals of speech recognition. At first, the speech corpus will be described, followed by an introduction to feature extraction and preprocessing. Further, the acoustic model and suitable training and decoding algorithms are presented. The last section characterizes the challenges of reverberation that are faced in speech recognition.

2.1 Speech Corpus

As a well structured speech corpus is a prerequisite for an ASR system, that requires training and test data, the Linguistic Data Consortium provides speech corpora tailored for speech recognition research. This thesis employs the WSJ0 Continuous Speech Recognition-corpus (November 1992), that consists of read speech from Wall Street Journal (WSJ) news texts. The advantage of this established corpus is, that it allows intra- and inter-group comparisons of research results. The corpus was designed by the DARPA Spoken Language System and consists of speech data (audio files and respective written text), different lexicons and language models to support research on a large-vocabulary continuous speech recognition (LVCSR) task. The speech data was captured by 2 microphones with a sampling rate of 16 kHz, a head-mounted noise canceling Sennheiser HMD414 and another one that might be of different kind. In addition to the labeling between the speech data and text, it contains continuous, as

well as spontaneous speech and includes equal portions of verbalized and non-verbalized punctuation. Information about the speaker and its sex is provided for each utterance, in order to do either Speaker-Dependent (SD) or Speaker-Independent (SI) training with variable amounts of data. The texts fall under either a 5000- or a 20,000-word subset of the corpus. In the context of this work, the SI-84 training data is used, which comprehends 12 hours of speech data (split into 7200 utterances) and SI-84 testing data containing 45 minutes (split into 333 utterances). To minimize the storage requirements for the corpora, the waveforms have been compressed using the SPHERE-embedded "Shorten" lossless compression format which was developed at Cambridge University. The NIST SPeech HEader REsources (SPHERE) software is included in a top-level directory and can also be used for decompression. Additionally, the corpus contains bigram and trigram language models for the 5000- and the 20,000-word subset with either verbalized- or non-verbalized punctuation. The whole corpus is provided by the LDC under the catalog number LDC93S6A [6]. For more details, we refer to [7].

2.2 Feature Extraction and Preprocessing

The recording of human speech with a microphone leads to a time-domain representation of the speech signal which contains more information than necessary for recognition of words or subword units. ASR means mapping a spectral envelope to a phone or a word [8]. Therefore, the goal of feature extraction is to extract the characteristic information and to reduce the dimension of the feature vector by removing redundant information [9].

2.2.1 Extration of MFCC-Features

The most popular speech features are the so-called Mel-Frequency Cepstral Coefficients (MFCCs). As illustrated in Figure 2.1, the discrete input signal $s(n)$, where n denotes the discrete time variable, is divided into overlapping frames by multiplying it with a Hamming window of fixed size [4]. Subsequent to this, the Discrete Fourier Trans-

formation (DFT) is computed for each frame. Since phase information is irrelevant to the human auditory system, it is reasonable to take the square magnitude of each frame's spectrum. To obtain the melspec coefficients $s_m(k)$, each frame is filtered by a mel-filter bank. Subsequently, the logarithm of the melspec coefficients is calculated to get the log melspec feature vector $s_l(k)$. The mel-filter bank has a better resolution for low than for high frequencies [9]. This is realized by triangular weighting functions for the mel channels, whose width increases with the channel number and approximates a logarithmic spectral resolution, motivated by the human auditory system [10]. However, as the frequency bands of the filter bank overlap, the components of the melspec coefficients are highly correlated [11]. Since the data will be used in a HMM based recognizer that expects diagonal covariance matrices, a cepstral transformation is required. So the last step to obtain the MFCCs $s_c(k)$ is to apply a Discrete Cosine Transformation (DCT) to the log melspec coefficient vectors to reduce their correlation [12].

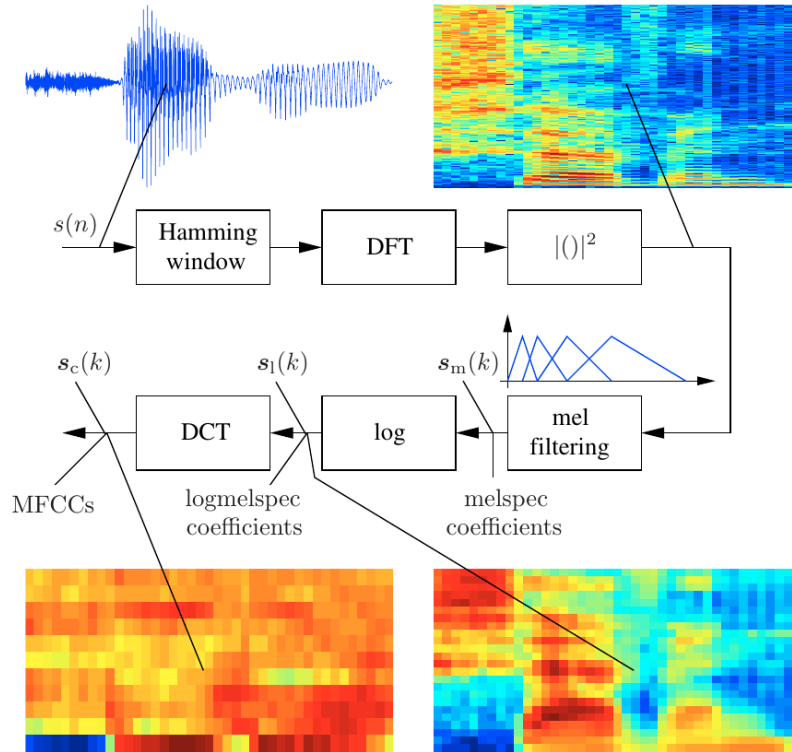


Figure 2.1: Illustration of the extraction of MFCC features [10].

In practice, speech is sampled at 8-22 kHz [9], windowed with a frame size of 25 ms and a frame shift of 10 ms [13]. Typically, the first 12 oder 13 components of a filter bank containing 24 channels suffice to differentiate between different acoustical parts of speech [9]. To increase the recognition accuracy, the first and second derivatives (named Δ - and $\Delta\Delta$ -features) can be concatenated to capture temporal context [12].

2.2.2 Feature-space Transformations

In order to improve recognition results or to handle challenging conditions as reverberation, feature transformations are often helpful. Within the scope of this thesis, a combination of LDA and MLLT as well as SAT and fMLLR is used. These common techniques are available in the trunk version of Kaldi that is used in this thesis.

LDA and MLLT

LDA is a linear feature transformation technique, that, in speech recognition, is usually applied on top of a MFCC+ Δ + $\Delta\Delta$ feature vectors. The aim is to reduce the feature vector's dimension and to improve the class discrimination between the classes of feature-vector components. Therefore, two scatter matrices, the between-class scatter matrix and the within-class scatter matrix can be used in several combinations to define an optimization problem [14]. The goal of LDA is to find a linear a transformation that maximizes the optimization criterion, by solving an eigenvalue problem. The transformation matrix is defined by eigenvectors corresponding to the largest eigenvalues [14]. Subsequent to the linear transformation of LDA, MLLT is applied to diagonalize the covariance matrices of the feature vectors. Hence, the objective of applying an MLLT is to minimize the loss in likelihood between full and diagonal covariance models [14]. The cascade of LDA and MLLT is verified by the invariance of LDA [14]. This results in a composite feature space of LDA+MLLT.

SAT and MLLR

The objective of SAT is to capture the variability in SI acoustic models attributed to phonetic variation, like differences in pronunciation and especially variations among the different speakers of the training data [15]. Speaker adaption using Maximum Likelihood Linear Regression (MLLR) comprehends adapting the acoustic model's parameters for each speaker from training data [15], whereas fMLLR directly transforms the acoustic feature vectors [16]. Performing MLLR, the acoustic model is an initialized SI model and separately adapted to each speaker in the training set by a linear regression transform [5]. To adapt mean and variance for each state, the transformation matrices are estimated independently so that they maximize the likelihood of the observed sequences [15, 17]. Modeling these speaker specific transformation is often computationally expensive and requires excessive storage [18]. Therefore, more economic methods for applying SAT on a large quantity of different speakers are presented in [18, 5]. To avoid this at all, fMLLR (a linear transformation in the feature-space), can be performed on top of MFCC or LDA+MLLT features. The transformation matrix is computed in an iterative way, using alignments from utterances to states of the acoustic model, computing the first and second order statistical moments, accumulating them and finally updating the estimated transformation matrix until a termination condition is satisfied [19]. The big advantage of this method is that there is no need to adapt the original acoustic model. Consequently, we can apply both presented methods [17].

2.3 Acoustic Model and Viterbi Training

The Hidden Markov Model (HMM) is a very popular stochastic approach to acoustic modeling of a Feature Vector Sequence (FVS), as it is possible to describe temporal and spectral variations [4]. Its task is, in conjunction with the language model, to determine the most likely word sequence given a FVS [8], precisely the probability $P(S = s|W = w)$ of observing a FVS S given the sequence of words W , that has actually been uttered [12]. HMMs can be considered as finite state machines, controlled

by two random experiments [4]. The first one determines the state sequence $q(k)$, hence the "hidden" transition from the previous state to the current state. The second random experiment determines the output feature vector according the output density $f_{\mathbf{s}(k)|Q(k)=j}(\mathbf{s})$, belonging to the current state $Q(k) = j$ [4]. Figure 2.2 shows a HMM with five states. A HMM λ is completely defined by the following parameters [4, 12]:

- $\mathcal{S}$, the set of states with cardinality $|\mathcal{S}| = N$
- α_{ij} , the state probability transitions
- $f_{\mathbf{s}(k)|Q(k)=j}(\mathbf{s})$, the multivariate output densities, usually modeled with Gaussian Mixture Models (GMMs)
- π , the initial state probabilities

Transitions may only be to the same state, to model different phone lengths, or to the next state, without the possibility of skipping a state [4, 8]. It is assumed, that the HMM starts in the first state and ends in the final state. Thus, the initial state probability has to be one for the first and zero for all other states.

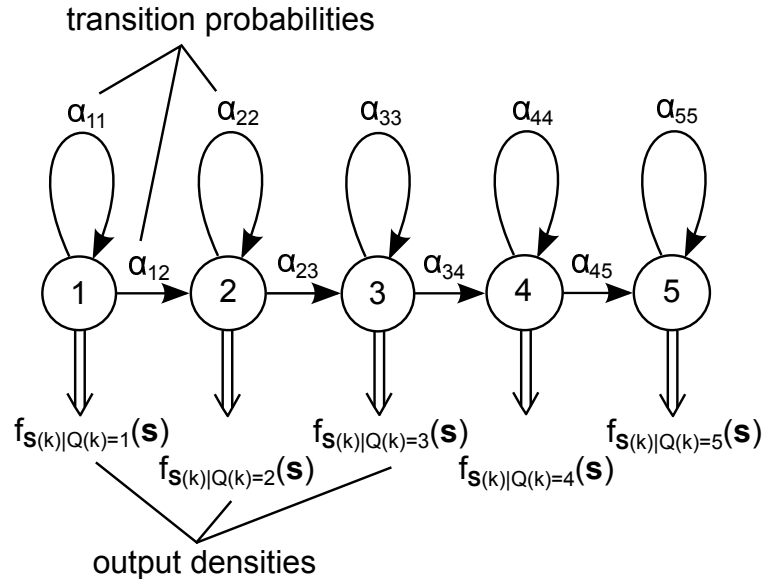


Figure 2.2: Typical HMM topology used in ASR [10].

The use of HMMs for speech recognition is based on certain assumptions [8]:

- The first-order Markov assumption implies that speech may be split into segments which are considered to be stationary.
- The independence assumption implies, that the probability of the current observation is only dependent on the current state and not on any previous states or previous output feature vectors.

The independence assumption is already a major simplification, since even clean-speech FVSs exhibit some statistical dependence and recorded speech is generally distorted by reverberation (which exhibits a stronger correlation between the feature vectors [10]). Several efficient algorithms for both training and decoding of HMMs have been developed, where the concepts of Viterbi training and Viterbi decoding are reviewed in the following sections.

Viterbi Training is an iterative training method to estimate the parameters of a HMM given a set of utterances with known transcripts. The objective is to adapt the parameters of the HMM, so that the probability of observing the FVS corresponding to the training utterances is maximized [10]. Given labeled training data, each feature vector is assumed to be produced by one specific HMM state. This is called alignment, since each feature vector is aligned to one HMM state. Figure 2.3 illustrates this proceeding by the green, blue and violet lines. Note, that as we handle a LVCSR task, words are divided into phones and each phone is modeled by one HMM state. Based on the alignment of feature vectors to HMM states, the Viterbi training comprehends on finding the most probable path, using the Viterbi algorithm presented in Section 2.4. Next, the HMM parameters on the path with highest probability are adapted using the Expectation Maximization algorithm [12]. This comprehends the transition probabilities, as well as the parameters of each HMM state's output density, represented by a GMM [12]. Viterbi training, in comparison to other training algorithms, reduces the computational cost significantly by considering the N most probable paths and finally adapting only the most probable path. Thereby it achieves similar recognition results

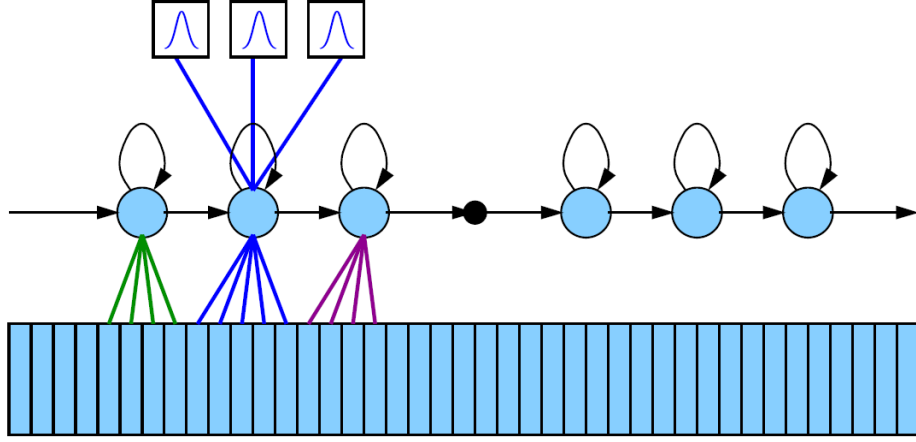


Figure 2.3: Illustration of the Viterbi training procedure [12].

if enough training data is available [12].

2.4 Viterbi Decoding

For decoding, a network of HMMs is constructed, which also incorporates information about word orders (grammar), a lexicon (containing words and their composition of sub-word units) and optional an n-gram language model [10]. Assigning a FVS to a sequence of HMM states can be seen as the task of finding the most likely path in this recognition network. The Viterbi algorithm can be applied to determine this path. In the following, the Viterbi score $V_i(k)$ (the likelihood of the most likely state sequence) is defined as [12]

$$V_i(k) = P(\mathbf{O}(1:k) = \mathbf{o}(1:k), Q(1:k-1) = q(1:k-1), Q(k) = i | \mathcal{M}). \quad (2.1)$$

The Viterbi score through the HMM network $\mathcal{M}$ ends in state $Q(k) = i$ at time k and belongs to the FVS $\mathbf{o}(1:k) = [\mathbf{o}(1), \dots, \mathbf{o}(k)]$ with $k = 1, \dots, K$. Each feature vector is of length M . To define the Viterbi algorithm, we define $B_i(k)$ as the most likely preceding state of state $Q(k) = i$ at time k and $q(1:k-1)$ is the state sequence that has been taken until time $k-1$. Furthermore, the output density $f_{\mathbf{O}(k)|Q(k)=i}(\mathbf{o})$ belonging to the FVS $\mathbf{o}(k)$ is denoted by $b_i(\mathbf{o}(k))$. In the following, the operator $\max_x f(x)$ finds the

maximum of $f(x)$, the operator $\arg \max_x f(x)$ finds the argument x , where $f(x)$ attains its maximum. Finally, the Viterbi algorithm can be split into four steps [20, 21]:

Step 1: Initialization

$$V_i(1) = \pi b_i(\mathbf{o}(1)), \quad i = 1, \dots, N \quad (2.2)$$

$$B_i(1) = 0 \quad (2.3)$$

Step 2: Induction

$$V_j(k) = \max_{i=1, \dots, N} [V_i(k-1)\alpha_{i,j}] \cdot b_j(\mathbf{o}(k)), \quad k = 2, \dots, K; \quad j = 1, \dots, N \quad (2.4)$$

$$B_j(k) = \arg \max_{i=1, \dots, N} [V_i(k-1)\alpha_{i,j}], \quad k = 2, \dots, K; \quad j = 1, \dots, N \quad (2.5)$$

Step 3: Termination

$$\text{The best score : } V_{max} = \max_{i=1, \dots, N} V_i(K) \quad (2.6)$$

$$\text{The most likely state at time K : } q^*(K) = \arg \max_{i=1, \dots, N} V_i(K) \quad (2.7)$$

Step 4: Backtracking

$$q^*(k) = B_{q^*(k+1)}(k+1), \quad k = K-1, K-2, \dots, 1 \quad (2.8)$$

$Q^* = [q^*(1), q^*(2), \dots, q^*(K)]$ is the most likely state sequence

The intermediate Viterbi scores, stored in a matrix, are calculated by multiplying each score of the possible predecessor states with the corresponding transition probability selecting the maximum among all these products. The final score for the most likely state sequence Q^* is found via backtracking and is obtained by multiplying the maximum product with the output density of the current state for the current feature given [10, 12]. Considering a LVCSR task, the high computational cost can be reduced by

only retaining the N most likely paths. This concept is known as pruning, where N denotes the pruning beam-width [22].

2.5 Challenges in Distant-Talking Speech Recognition

This section aims to give an insight about the effects of reverberation. Figure 2.4 shows a typical distant-talking ASR scenario.

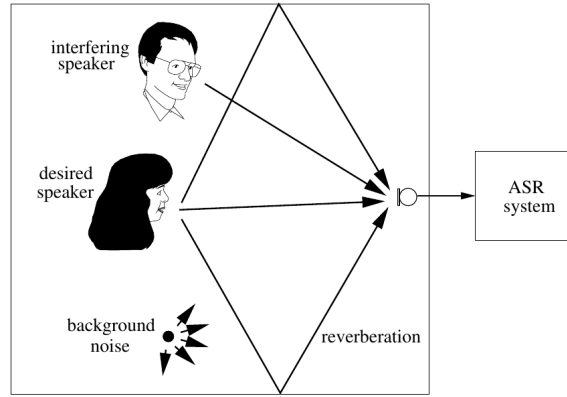


Figure 2.4: Distant-talking ASR scenario [4].

Besides the clean speech signal $s(n)$ traveling on the direct path to the microphone, the microphone may pick up different sources of noise: uncorrelated additive interferences denoted by $b(n)$, like background noise or competing speaker, as well as many delayed and attenuated copies of the desired signal[4], originated from reflections of sound waves by walls and other objects within the room [4]. These multi-path propagations are known as reverberation. The microphone signal $x(n)$ can be modeled by convolving the clean time-continuous speech signal $s(n)$ with the impulse response $h(n)$ (which models the acoustic wave propagation) [10]:

$$x(n) = s(n) * h(n) + b(n). \quad (2.9)$$

Note, that this thesis focuses on reverberation, so the additive interferences can be

neglected:

$$x(n) = s(n) * h(n). \quad (2.10)$$

An example of an acoustic environment $h(n)$, more precisely a Room Impulse Response (RIR) measured with a sampling frequency of $f_s = 16$ kHz is shown in Figure 2.5.

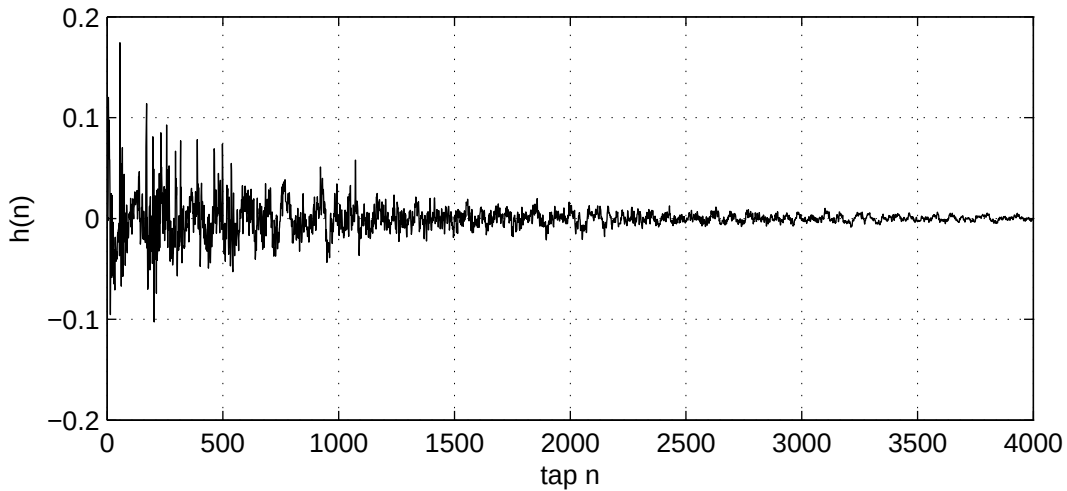


Figure 2.5: Example of a RIR $h(n)$.

The first peak, caused by the direct-path sound waves is followed by several distinct peaks, corresponding to prominent reflections [4]. With increasing delay, more and more weak overlapping reflections establish an exponentially decaying envelope describing the expiration of the RIR [4]. Therefore, the length of a RIR is infinite in theory, even so, the time needed for a 60 dB decay in sound energy is called the reverberation time T_{60} [4]. Typically, reverberation time ranges from 200 ms to 800 ms in offices or living rooms, but can reach values over 1 s in large halls [4].

As aforementioned, reverberation can be modeled as a convolution of the clean speech signal $s(n)$ and the RIR $h(n)$. This essentially affects the statistical properties of the feature vector which leads to a mismatch between the acoustic model and the observed input data [4], especially, if the ASR system is trained on clean, non-reverberant speech data and tested on reverberant data [12]. Figure 2.6(a) shows the MFCC FVS of a

close-talking recording of the digit sequence "four, two, seven", uttered by a female speaker [10]. To illustrate the effect of reverberation, the differences between the clean FVS in Figure 2.6(a) and the associated reverberated FVS in Figure 2.6(b) are pointed out in more detail [10]. Since MFCCs can be considered as the Fourier transformed coefficients of the log melspec vector, the 0-th MFCC channel corresponds to the logarithmic frame energy, the others represent the spectral changes over the mel channels [10]. The 0-th channel exhibits high values for all vowels and low values for consonants [10]. As mentioned in Section 2.2, the coefficients are decorrelated by the DCT. So the dispersive effect of reverberation can mostly be identified by comparing the 0-th channel of the clean and the reverberant FVS, where the energy is smeared over the time axis [10].

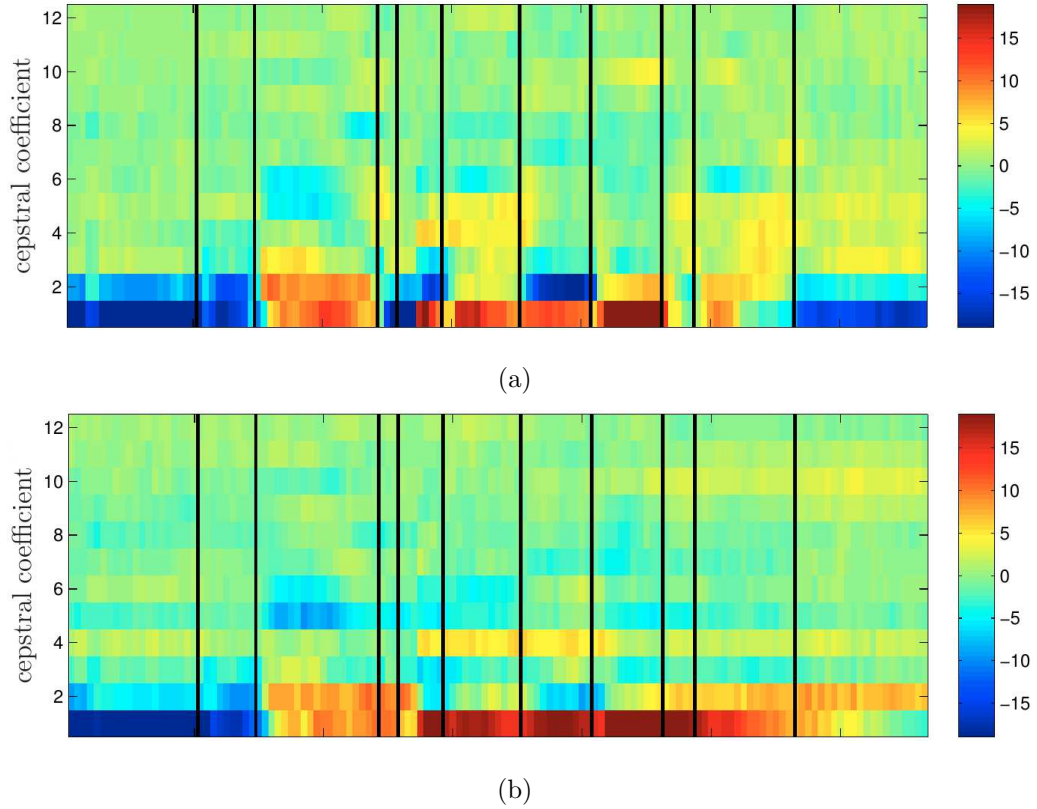


Figure 2.6: MFCC representation of the utterance "four, two, seven": a) FVS of a clean speech signal, b) FVS of a reverberated speech signal [10].

3 Kaldi Toolkit

The following chapter shall give some overall information about the Kaldi toolkit. Afterwards, the principle of WFSTs and their application for graph creation in Kaldi will be introduced.

3.1 General Overview of Kaldi

The Kaldi speech recognition toolkit originated from a project at Johns Hopkins University. The processing aim for Kaldi was quite similar to the capabilities of the common HTK Toolkit, but different in its composition [23]. With the objective of developing modern, flexible general-purpose software, it was first released in 2011, but, also as it's purpose is to access the speech-recognition research community, it is in a constant developing process [24]. The Kaldi toolkit is open source to encourage researchers to modifications and extensions. Development was focused on a structured, generic code, additionally taking advantage of the inclusion of open source libraries, what discriminates Kaldi from other common ASR softwares. The application of Finite State Transducers is realized by including the OpenFst library [24]. Kaldi differs from HTK in using the Viterbi algorithm for training and decoding to achieve similar results though faster (see Section 5.1). Furthermore, the Kaldi Toolkit provides codebases for scientific research on acoustic modeling based on deep neural networks [25]. However since Kaldi is quite new, the manuals focus on how to apply the existing code without offering a detailed documentation like the HTK Book (see [11]). This is the main motivation for introducing the functionalities of the Kaldi Toolkit in more detail. The Kaldi

toolkit compiles on common Unix-like systems and on Microsoft Windows [24]. Two major version are available on SourceForge (see kaldi.sourceforge.net): a trunk version containing the latest code updates and that was used under the scope of this work and a stable version, that is a subset of the trunk version, containing the thoroughly tested code. These contain complete recipes for building speech recognition systems.

3.2 Speech Recognition with Weighted Finite-State Transducers

This section aims to give an overview of the use of WFSTs in speech recognition. These transducers can be applied, as they provide a common and natural representation for major components of speech recognition systems, such as HMMs, context-dependency models, n-gram statistical language models, pronunciation dictionaries (lexicons), statistical grammars and word or phone lattices [26]. General algorithms and operations that support building and optimizing WFSTs are also presented briefly. The following contents refer to [26].

Weighted Finite-State Transducers

WFSTs consist of a set of initial states with initial weights, a set of final states with final weights and a set of transitions between the states. The transitions are labeled with an input symbol, an output symbol and a weight (likelihood). The output string of a path is specified concatenating the outputs of the states passed, the overall probability for a path is given by the product of the passed transition probabilities. Figure 3.1(a) shows an example of a WFST (representing a language model), giving each transition identical input and output labels. Figure 3.1(b) represents a toy pronunciation lexicon, that is used to transduce phone strings to words, in this example "data" and "dew", with probabilities representing the likelihoods of alternative pronunciations. The transition, whose input is the first phone of a word outputs the word. This is followed by

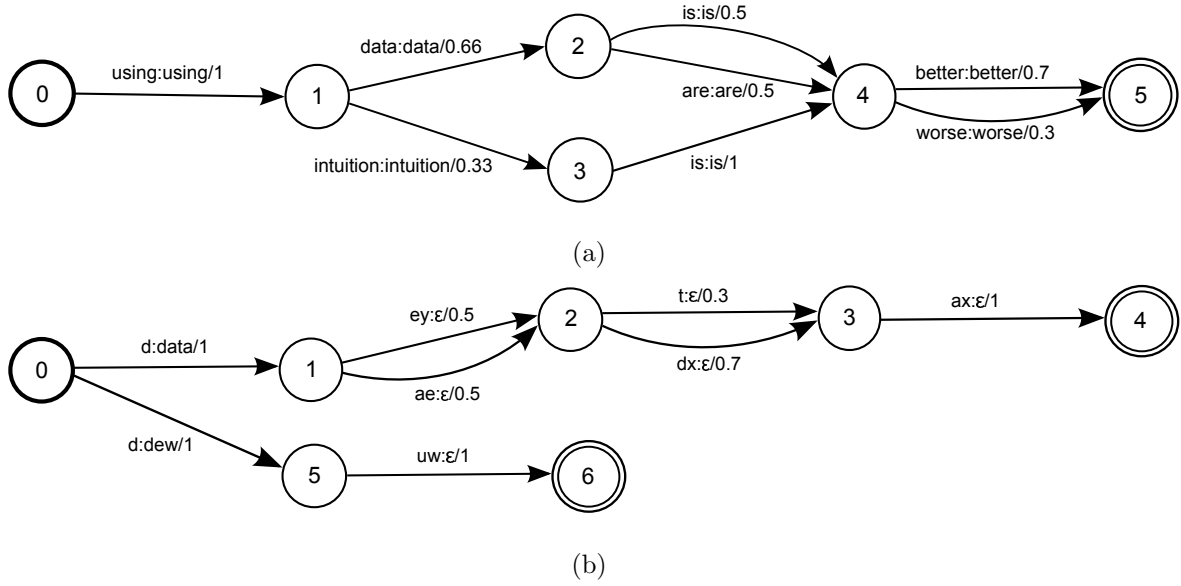


Figure 3.1: Examples of WFSTs representing a) a language model and b) a lexicon [26].

transitions, that output an ϵ label, because the output was already determined. Generally, an ϵ label denotes a transition that either consumes no input or that produces no output. The chief advantages of WFSTs is the possibility of combining models: since transitions have an input and an output label, it is possible to combine more than one transducer without losing output identity. That implies that the transducer can represent a relationship between two levels of representation, for instance between phones and words, or between HMMs and context-independent phones.

Operations on Weighted Finite-State Transducers

The following section presents common methods for combining and optimizing WFSTs. These mathematical operations lead to a simple and flexible ASR decoder design.

Composition, denoted by $\circ$, is the transducer operation for combining different levels of representation, that works for both, context-independent and context-dependent transducers. The relational composition of two transducers T_1 and T_2 is denoted by

$$T = T_1 \circ T_2. \quad (3.1)$$

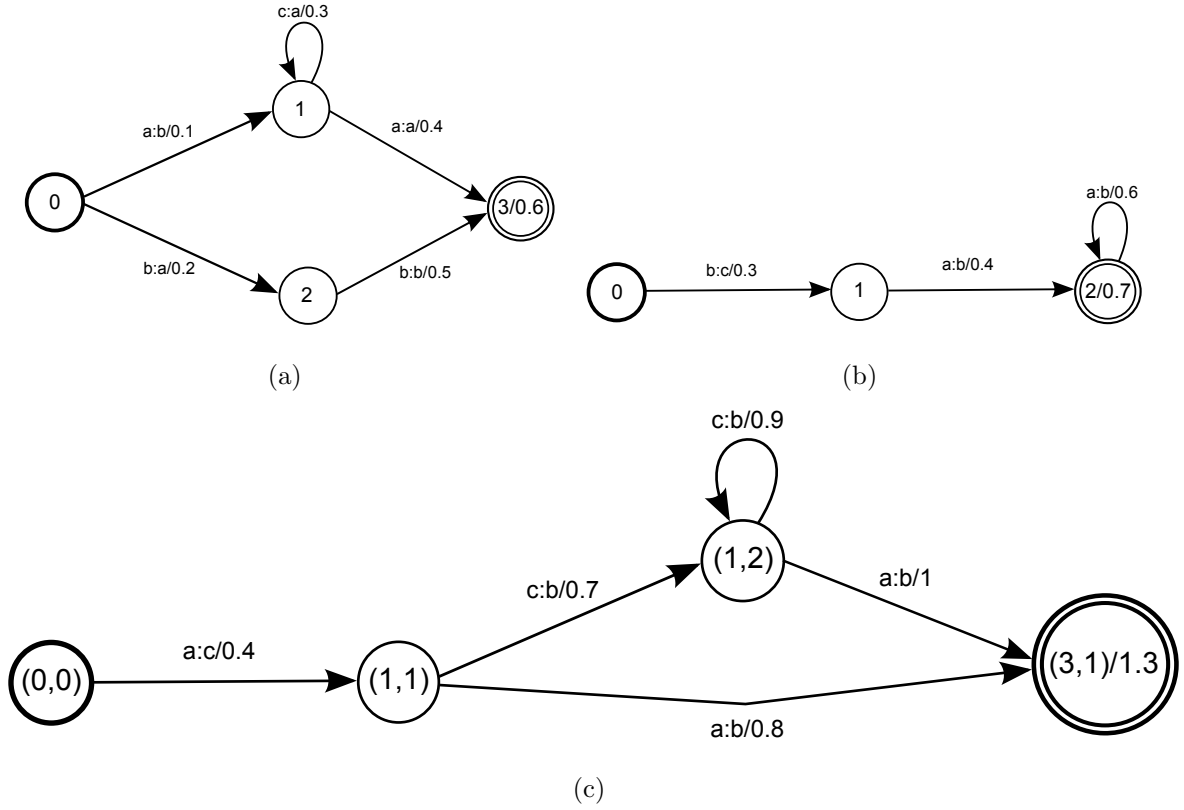


Figure 3.2: Example of the composition of two transducers a) and b) [26].

In particular, the operation introduces one path mapping string u from T_1 to some intermediate string v , that then maps v to w from T_2 . The weight of this path in T is computed from the weights of the two corresponding paths in T_1 and T_2 . Figure 3.2 shows an example for a composition of two transducers. T satisfies the following conditions:

- Its initial state is the pair of the initial state of T_1 and the initial state of T_2 .
- Its final states are pairs of the final state of T_1 and the final state of T_2 .
- There is a transition t from q_1, q_2 to r_1, r_2 for each pair of transitions t_1 from q_1 to r_1 and t_2 from q_2 to r_2 such that the output label of t_1 matches the input label from t_2 .

That is the transition t takes its input label from t_1 , output label from t_2 and its weight is the combination of the weights t_1 and t_2 . If the transition weights are probabilities, the operation is the product, if the weights are log probabilities or negative log probabilities, the operation is the sum. Transitions with ϵ -labels in T_1 or T_2 must be treated specially (see [26]).

Determinization, denoted as *det*, of a transducer means, that states don't have more than one transition with the same input label and there are no input ϵ -labels. This operation is applied to remove redundancy, what reduces the time and memory requirements and improves the decoding speed. To eliminate redundant paths, weighted determinization needs to calculate the combined weight of all paths with the same labeling. The combined weight for a set of paths would be the product (or the sum) of the weights of the paths. The opportunity of keeping only the most probable paths is called Viterbi approximation. Transitions with the same input label can have different weights, but only the minimum of those weights is needed, as the most probable transition is chosen. The determinization algorithm transforms a non-deterministic transducer into an equivalent deterministic transducer. Two weighted transducers are referred equivalent, if they associate the same output string and weights, including the final weight, to each input string. An example for the determinization of a WFST is shown in Figure 3.3. Applying the algorithm, the resulting transducer might be tree-shaped, but gives also results for transducers containing cycles, which else have no tree representation.

Given a deterministic weighted automaton, its size can be reduced by *minimization*, brief *min*. This operation can efficiently save both space and time. Any deterministic weighted transducer can be minimized. The resulting weighted transducer is equivalent to the input transducer, but has the least number of states and transitions compared to all transducers equivalent to the input transducer. An example for minimization of WFSTs is shown in Figure 3.4.

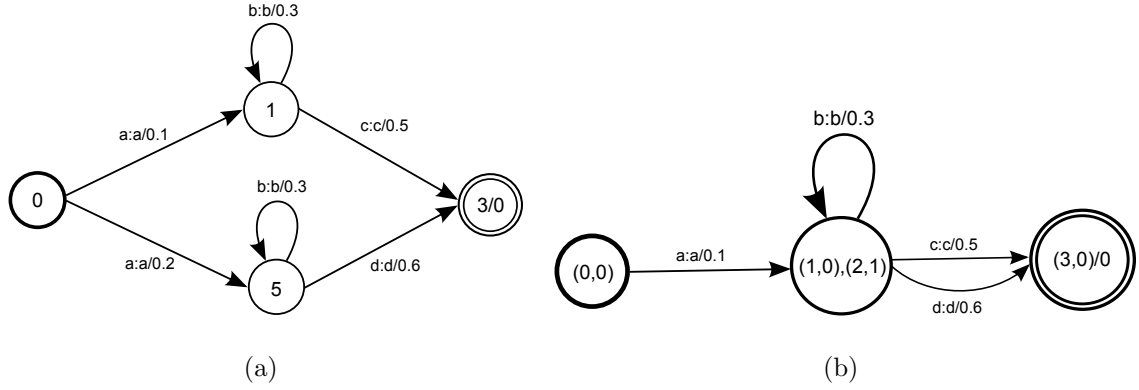


Figure 3.3: Example of the determinization of WFSTs: a) a nondeterministic transducer and b) an equivalent deterministic transducer [26].

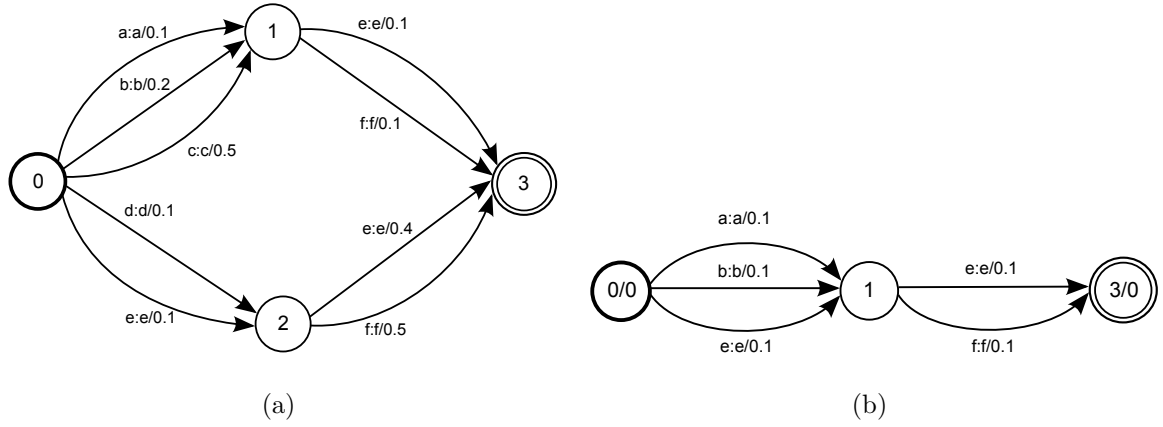


Figure 3.4: Minimization of WFSTs: a) a deterministic transducer and b) an equivalent minimal transducer [26].

3.3 Graph Creation in Kaldi

This section describes the creation of the decoding graph in Kaldi with the goal to map feature vectors to context-dependent phones and words. The approach is similar to common recipes, but differs in some details. The combined and optimized decoding graph includes the following processing steps [27]:

$$HCLG = asl(min(rds(det(H' \circ min(det(C \circ min(det(L \circ G))))))). \quad (3.2)$$

The graph construction (following the Equ.(3.2)), is executed from right to left. First, the union of the lexicon L (mapping phones to words), and the grammar G (a language model acceptor, whose input and output symbols are words), are composed. The intermediate result is optimized applying determinization and minimization. Next, we want a transducer, that maps from context-independent phones to context-dependent triphones, what means that left and right state-neighbors are taken into consideration. Therefore, a context-dependency transducer is composed from the left. The result is again optimized and afterwards composed with H' . Here H is the HMM transducer, whose input symbols are transition-ids and its output symbols are context-dependent phones and the $'$ denotes that self-loops are removed. The result is finally optimized, including *rds*, which means "remove-disambiguation symbols" and *asl* " add-self-loops". The aim was to ensure that at every stage of graph creation it will be stochastic, meaning that all state's weights sum to one [27]. For this, it is assumed that if the transducer in the precedent stage was stochastic, it will also be in the next stage. This involves also, that each operation has to preserve stochasticity. In order to keep the decoding graph determinizable, *disambiguation symbols* are inserted (and removed later). Disambiguation symbols are numbering symbols, that are inserted at different points. In the lexicon, the end of phone sequences is denoted if the sequence appears for example in more words to ensure uniqueness. Furthermore, disambiguation symbols are inserted in the context-dependency model C to describe the beginning and end of utterances and in the language model. The operation *asl* is performed whenever during composition the transducer on the right has disambiguation symbols on its input. Consequently, a self-loop is added in the left transducer for each of the disambiguation symbols [27].

4 Speech Recognition System

This chapter presents the speech recognition system itself, thus the files and folders this includes and the particular steps the setup comprises. We refer to the trunk version of the Kaldi Toolkit (Revision 3511) and appendant standard functions. Further, section 4.2 describes, how the synthesization of reverberated speech data based on measured RIRs was performed.

4.1 Speech Recognition Pipeline

In the following section, we will explain the single steps of the script `run.sh`, which is part of the standard Kaldi recipe and placed in the directory `kaldi-trunk/egs/wsj/s5`. The directories described in the following parts are subfolders of this main directory. In the following descriptions, Kaldi uses bash scripts ending in `".sh"` to call C++ files ending in `".cc"`. Also note for all following sections, that in Kaldi, marks are inserted into certain files for clear identification. This concerns speaker Identifiers (IDs), utterance IDs and especially transition IDs, to have one integer value for every transition between two specific states.

At first, the Kaldi Toolkit can be configured to run as a cluster of Linux machines by a GridEngine or locally [28]; if you run Kaldi on the local computer, it is important to make sure to choose the option number of jobs `"- -num-jobs"`, especially for decoding, not higher than the number of CPUs on the respective computer.

Data Preparation

The following command line extracts the training set `train_si84`, the test set `test_eval92` and a 5,000-word subset from the test set `test_eval92_5k` from the speech corpus:

```
local/wsj_data_prep.sh $wsj0/??-?,??.? $wsj1/??-?,??.?
```

Several lists are extracted from the speech corpus. Some belong to a particular dataset, others are equal for all data. All files are stored in `data/local/data` (see Figure 4.1):

- "dataset".txt maps utterance IDs to the text spoken
- "dataset".spk2utt assigns each utterance to speaker IDs
- "dataset".utt2spk lists the utterance IDs and the corresponding speaker IDs
- spk2gender denotes all the speaker IDs and its gender
- "dataset"_wav.scp contains the absolute paths to each utterance, stored in the `wv1` format, but including a command to transform them into `wav` format

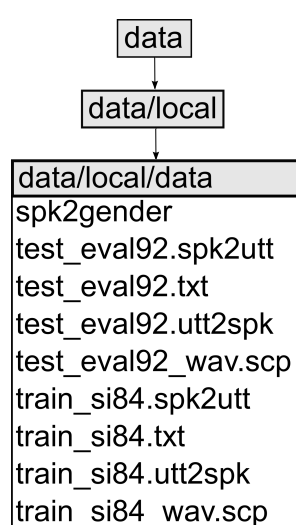


Figure 4.1: Overview of the generated files and folders during data preparation.

Dictionary Preparation

The following command line downloads the public CMU dictionary from this link:
<https://svn.code.sf.net/p/cmusphinx/code/trunk/cmudict>.

```
local/ws_j_prepare_dict.sh
```

The files mentioned below are stored in the directory `data/local/dict`:

- `lexicon.txt` contains 133338 words and the corresponding phone sequence
- `nonsilence_phones.txt` contains the 39 phones of the WSJ corpus
- `silence_phones.txt` describes the three silence phones of the WSJ corpus

Lexicon Preparation

This script prepares the directory `data/lang`, given the data from directory `data/local/dict`:

```
utils/prepare_lang.sh data/local/dict "<SPOKEN_NOISE>" data/local/lang_tmp  
data/lang
```

These files were created in order to capture different stress or tone variations and can be found among others in `data/lang/phones`. One basic part of this command-line is the addition of word-position markers for (B)egin, (E)nd, (I)nternal, and (S)ingleton to phones:

- `align_lexicon.txt` contains words and corresponding phones, where word-position markers were added
- `nonsilence.txt` contains all non-silence-phones with word-position markers
- `silence.txt` contains silence phones with word-position markers
- `word_boundary.txt` contains all phones with their possible word-position marker
- `disamb.txt` contains all disambiguation symbols

- roots.txt and sets.txt itemize vowels and their corresponding stress-dependent realizations
- extra_questions.txt is a list of phones with the same stress

From the basis of the lexicon.txt, a list lexiconp.txt is generated that adds the appropriate pronunciation probabilities to words. These alignments between words and phones are then used to determine the WFST for the lexicon L.fst that describes the probabilistic relationship between words and phones. L.fst and L_disamb.fst are stored in the directory data/lang. Besides, the HMM topology is determined for non-silence and silence phones in the file topo. Non-silence phones are represented by three states per HMM, silence phones by 5 states per HMM. Figure 4.2 overviews all files and folders denoted in this section.

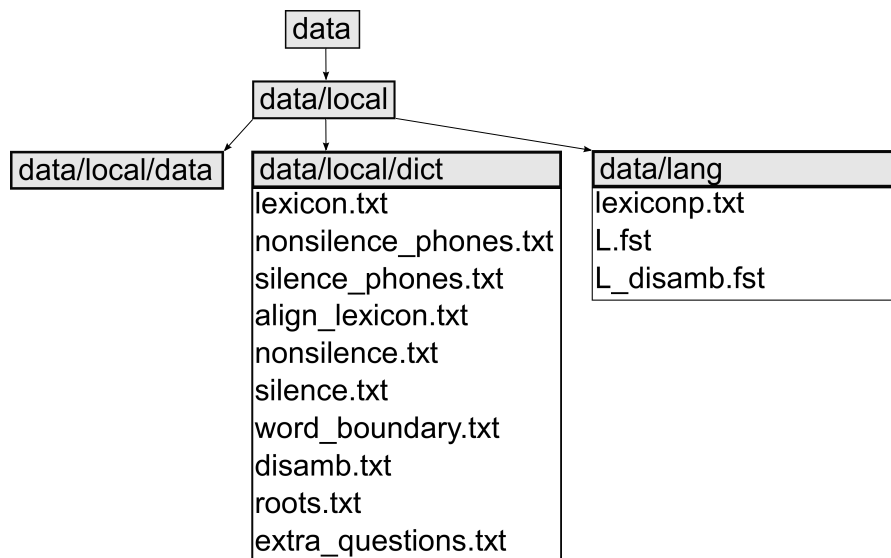


Figure 4.2: Overview of the generated files and folders during lexicon preparation.

Data Formatting

The following command prepares training and test data and several language models G.fst:

```
local/ws_j_format_data.sh
```

This includes copying already existing information about the datasets, such as text, spk2utt, utt2spk, spk2gender and the wav.scp file. These have to be distributed then to the appropriate folders `data/train_si84` and `data/test_eval92`. The language models, that were already stored in `data/local/nist.l` are unpacked and stored accordingly in the directory `data`:

- `lang_test_bg` is a 20,000 word bigram language model
- `lang_test_bg_5k` is a bigram language model for a 5,000 word subset
- `lang_test_tg` is a 20,000 word trigram language model
- `lang_test_tg_5k` is a 20,000 word trigram language model for a 5,000 word subset
- `lang_test_tgpr` is a pruned 20,000 word trigram language model
- `lang_test_tgpr_5k` is a pruned trigram language model for a 5,000 word subset

Every folder contains the language model `G.fst`, the lexicon `L.fst` and a numbered list of all phones `phones.txt` and words `words.txt` and the complete directory phones described above. See Figure 4.3 for an overview.

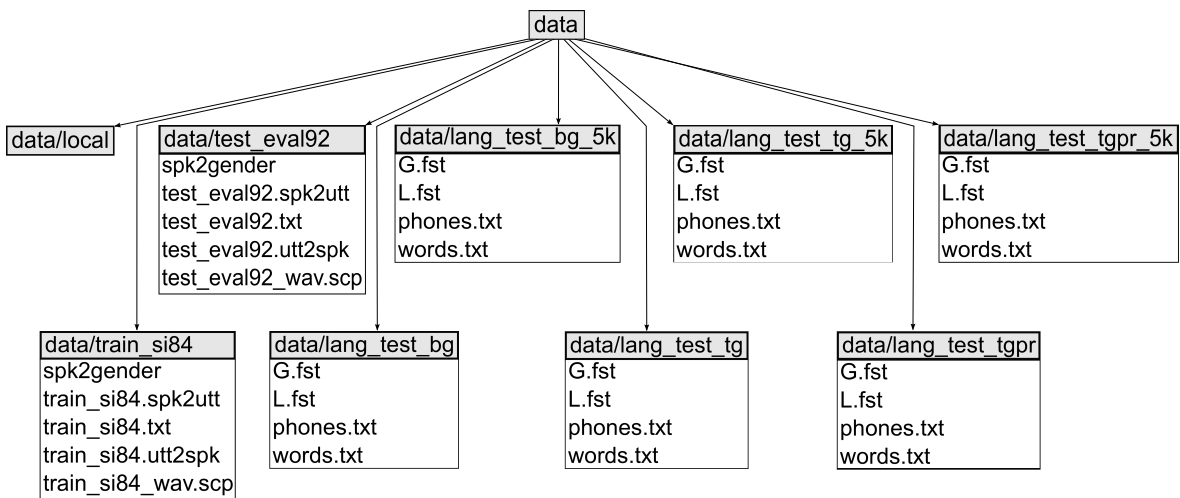


Figure 4.3: Overview of the generated files and folders during lexicon preparation.

Feature Extraction

This section describes, how the MFCC-feature extraction is executed:

```
mfccdir=mfcc
for x in test_eval92 test_eval92_5k train_si84; do
steps/make_mfcc.sh - -cmd "$train_cmd" - -nj 20
data/$x exp/make_mfcc/$x $mfccdir;
steps/compute_cmvn_stats.sh data/$x exp/make_mfcc/$x $mfccdir;
done
```

In the first line, we define the output directory. The commands are executed for all datasets denoted in the second line. The function `make_mfcc.sh` creates the MFCC-features. Inputs to the function `compute-mfcc-feats.cc` are the file `wav.scp`, an `rspecifier` and a `wspecifier`, that are strings that specify the directories to read from or to write into. Next to the default settings, the function offers options like DC offset removal, minimum and maximum frequency cutoffs, Vocal Tract Length Normalization, or the choice of frame shift with corresponding windowing function can be chosen (see [29]). Note that Δ - or $\Delta\Delta$ -features are not produced by this command. The output files are compressed archives in the specified directory and a file `feats.scp` for each dataset (in the directory `data/"dataset"`), assigning the archives to the corresponding utterances.

In addition to feature extraction, Kaldi offers the function `compute_cmvn_stats.sh`, which computes Cepstral Mean and Variance Normalization (CMVN) statistics per speaker. The output files are, similar to the MFCC feature extraction, compressed archives in the directory `mfcc` and `cmvn.scp` files for each data set stored in `data/"dataset"`, assigning the archives to the corresponding utterances from a certain speaker. Figure 4.4 shows an overview of all files folders denoted in this section.

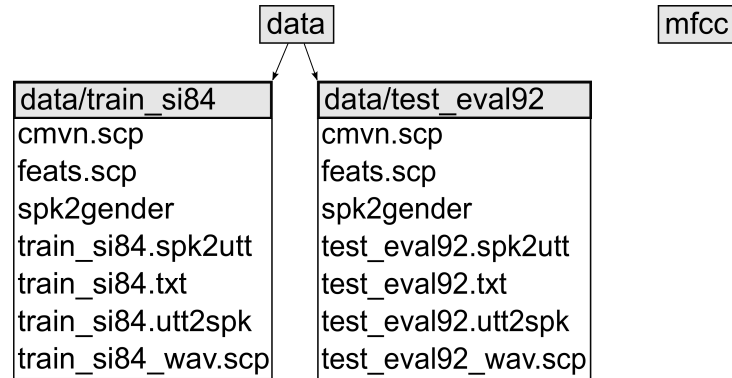


Figure 4.4: Overview of the generated files and folders during feature extraction.

Monophone Training and Graph Initialization

This command performs monophone training:

```
steps/train_mono.sh - -boost-silence 1.25 - -nj 10 - -cmd "$train_cmd"
data/train_si84_2kshort data/lang exp/mono0a
```

The first input is the folder `data/train_si84_2kshort`, which is a subset of the 2,000 shortest utterances from the `train_si84` data and includes the MFCC-features, the CMVN statistics as well as utterance to text alignments. The second input variable is the directory `data/lang`, which contains the lexicon (the alignment between phones and words), the HMM topology and information about the phones as described in Section 4.1. The third input defines the output directory `exp/mono0a`. The monophone training can be executed by setting the parameters of the pruning beam width of the Viterbi training, the number of iterations, the total number of Gaussian distributions and the increase of the number Gaussian distributions per iteration step. The Viterbi training for monophones includes the following processing steps:

- `add-delta.cc` adds Δ - and $\Delta\Delta$ -features to the MFCC-features.
- `gmm-init-mono.cc` is applied to initialize the GMMs for each phone's states. The function outputs the model `0.mdl` containing HMM topology information and the decision tree that models context-dependency for monophones.

- `compile-train-graphs.cc` compiles one WFST `fsts."job-number".gz` for each utterance using the tree, the model, the lexicon and the utterance to text alignments
- `align-equal-compiled.cc` creates the initial alignments `ali."job-number".gz` between an utterance and the corresponding sequence of transition-IDs. Considering these alignments allows us to update the parameters of the HMM, thus to maximize the probability for the best path. Alignments will be explained in more detail in Section 4.1.
- `gmm-acc-stats-ali.cc` accumulates the GMM parameters, inserts the feature vectors according to the alignment and updates the parameters of the state sequence, thus increases the likelihood for the most probable paths.
- `gmm-est.cc` uses the accumulated GMM parameters and applies Maximum-Likelihood re-estimation to the HMM parameters and updates all parameters.

Subsequently, only the following steps will be repeated in each iteration:

- `gmm-align.compiled.cc` searches the best path in each utterance's WFST to get a new alignment
- `gmm-acc-stats-ali.cc` accumulates the GMM parameters of the state sequence
- `gmm-est.cc` updates the transition probabilities and the GMM parameters for the best path

The output will be stored in `exp/mono0a`. Errors will go to the log files `exp/mono/log`. For better comprehension, Figure 4.5 overviews all files and folders mentioned in this section.

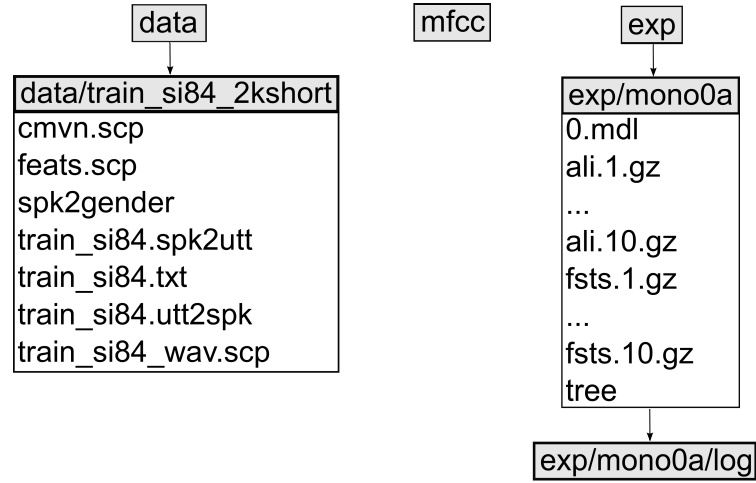


Figure 4.5: Overview of the generated files and folders during monophone training.

Graph Creation

This command compiles a WFST decoding graph according to the HCLG phrase (see Section 3.3) and stores the results in `exp/mono0agraph.tgpr`:

```
utils/mkgraph.sh - -mono data/lang_test.tgpr exp/mono0a exp/mono0a/graph.tgpr
```

The function `fsttablecompose.cc` composes the lexicon `L_disamb.fst` and the language model `G.fst`. The result is then determinized, minimized and proved to be stochastic. Next, `fstcomposecontext.cc` composes the WFST LG with a context dependency model, but for monophone decoding, we use context-dependency of context-window size one, thus no context-dependency. Afterwards, the WFST is again determinized, minimized and proved to be stochastic. The function `make-h-transducer.cc` creates the HMM WFST `H.fst` using the transition ids, the monophone tree and the trained HMMs from `exp/mono0a/final.mdl`. The file `H.fst` then has to be integrated by `fsttablecompose.cc` to obtain the complete HCLG decoding graph. The result is finally determinized, minimized, proofed to stochastic and then stored in `exp/mono0a/graph.tgpr/HCLG.fst`.

Decoding

After graph creation, the decoding can be started with the command:

```
steps/decode.sh -nj 8 -cmd "$decode_cmd" exp/mono0a/graph_tgpr
data/test_eval92 exp/mono0a/decode_tgpr_eval92
```

The function `gmm-latgen-faster.cc` uses the decoding graph to generate a list of the N best paths, known as lattice, that is stored in `exp/mono0a/graph_tgpr/lat."job-number".gz`. The function `score.sh` is then used to find the most probable path in the lattice to specify the word error rate (WER).

Alignments

Generally, an alignment represents a mapping from an utterance to a sequence of transition IDs. As transitions can be unambiguously identified by its corresponding ID, they are used as input labels for decoding graphs to map a feature vector to a phone or to a particular transition. The goal is to align the N feature vectors extracted by an utterance to its corresponding N transition ids. This alignment can then be used to update the parameters of the HMM by considering the best path. The required alignments can be produced like this:

```
steps/align_si.sh -boost-silence 1.25 -nj 10 -cmd "$train_cmd" data/train_si84
data/lang exp/mono0a exp/mono0a.ali
```

Further training methods apply either context-dependency using triphones, or other training utterances. Here, the inputs are the whole training data `data/train_si84`, the lexicon in `data/lang` and the previously generated training alignments `final.mdl`. The script `steps/align_si.sh` can be executed either using the option `-use-graph true`, or not. If you use it, the function `gmm-align-compiled.cc` searches the best path for the MFCC FVS in the already given training alignments. The results can be found in `exp/mono0a.ali/ali."job-number".gz`. If you do not use the option (`-use-graph false`), the script `steps/align_si.sh` will use the tree, the file `final.mdl` and the lexicon `L.fst` to calculate alignments for the test utterances. The alignments produced by the function

steps/align_si.sh are valid for MFCC+ Δ + $\Delta\Delta$ - and LDA-features.

Training with context dependency

In the previous described processing steps for monophone training and decoding, context dependency was not yet considered. This command performs the incorporation of context dependency C into the HCLG-decoding graph:

```
steps/train_deltas.sh - -boost-silence 1.25 - -cmd "$train_cmd" 2000 10000
data/train_si84_half data/lang exp/mono0a.ali exp/tri1
```

The script can be carried out taking some employments like the pruning beam-width, the number of iterations, or the total number Gaussian distributions. The integration of context-dependency is realized by clustering the phones in so-called phonetic questions according to their acoustic similarity. Phonetic clustering includes accumulation of statistics by using `acc-tree-stats.cc`, clustering of the phones by `cluster-phones.cc` and compiling the questions by `compile-questions.cc` to obtain the file `exp/tri1/questions.qst`. This is used to create a phonetic decision tree `exp/tri1/tree` by using `build-tree.cc` that incorporates context-dependency in a specific context window, specifically triphones. The function `gmm-init-model.cc` employs the new decision tree to initialize the GMMs and stores it in `exp/tri1/1mdl`. The new decision tree and the new model are now used to create new alignments with the function `convert-ali.cc`. These are stored in `exp/tri1/ali."job-number".gz`. By now, the proceeding is even to the monophone training. We use the functions `compile-train-graphs.cc`, `gmm-align-compiled.cc`, `gmm-acc-stats-ali.cc` and `gmm-est.cc` to train the acoustic model and obtain `exp/tri1/final.mdl`, `exp/tri1/ali."job-number".gz` and `exp/tri1/fsts."job-number".gz`.

Triphone Graph Creation and Decoding

Subsequently, the command to determine the fully expanded decoding graph is similar to the one in section 4.1, except the option `"- -mono"` is not applied.

```
utils/mkgraph.sh data/lang_test_tgpr exp/tri1 exp/tri1/graph_tgpr
```

Consequently, also the same steps as in Section 4.1 are applied, except for creating CLG, a triphone context-dependency model C is input to the function `fstcomposecontext.cc`. The command for decoding did not change, as the input parameters are the same, but were stored in different folders in the previous steps.

```
steps/decode.sh -nj 10 -cmd "$decode_cmd" exp/tri1/graph_tgpr
data/test_eval92 exp/tri1/decode_tgpr_eval92
```

4.2 Creation of Reverberated Test Data

Under the scope of this work, recognition accuracy is also investigated for reverberated data. This section describes, how the reverberation was performed. As already denoted in Section 2.5, reverberation can be modeled by convolving a clean speech signal with a RIR that models the acoustic environment. Table 4.1 sums up the characteristics of the five rooms, whose RIRs were used to reverberate the test data.

	A	B	C	D	E
T_{60}	300 ms	600 ms	700 ms	780 ms	900 ms
d	2.0 m	2.0 m	4.1 m	2.0 m	4.0 m
N	18	18	6	18	22

Table 4.1: RIR characteristics of five rooms: T_{60} is the reverberation time, d is the distance between the acoustic source and the microphone, N is the number of RIRs measured for the room [12].

At first, all the test data has to be converted from the compressed ".wv1-format" to the ".wav-format", so that it can be processed with Matlab. The Matlab script for the reverberation loads the impuls responses, that are stored in .mat files. All 333 test utterances are then convolved with one of the N RIRs in the ".mat-file". A Matlab function is applied, which realizes the convolution as multiplication in the frequency domain by applying the Fast Fourier Transform. Finally, the Matlab function

"audiowrite" is used to write ".wav-files". In addition, a list containing the absolute path to all files is created. That is required for using the reverberated test data, because the already existing list contains a command for wv1-to-wav conversion, that can not be used anymore, because the files are already in ".wav-format". Finally, the reverberated data is used input of the feature extraction described in Section 4.1.

5 Experimental Results

This section shall give an insight to the recognition accuracy we investigated with Kaldi using the WSJ corpus of November 1992 (described in Section 2.1). All recognition results are specified by the metric Word Error Rate (WER) in percent [30]. The following recognition results are gained using only clean training data and recognition results will be presented for clean and reverberated test data. All experiments were performed on the local CPU, an Intel(R) Core(TM) 2 Quad CPU Q9950 @ 2.66 GHz and 8 GB RAM.

5.1 Performance Comparison between Kaldi and HTK

In this section, we compare the performance of Kaldi and the HTK Toolkit for undistorted test data by means of Table 5.1. The recognition rates generated with Kaldi include triphone context-dependency and Δ - and $\Delta\Delta$ -features were appended to the feature vectors. For the decoding graph creation we used a bigram language model. The pruning beam-width was chosen to 6 in each step. The recognition results from the HTK setup is an implementation of the AURORA 4 recognizer investigated in [12]. It uses a bigram language model and 3-state triphone HMMs for non-silence phones with continuous 16-component GMM output densities. Therefore, HTK used a total number of 2500 Gaussian. In Kaldi, we chose the total number of Gaussians to 10,000, as we consider context-dependency.

	HTK	Kaldi
WER	10.70	14.66
$Time$	3 h 25 min	1 h 12 min

Table 5.1: Comparison between Kaldi and HTK in terms of performance (WER) and computation time ($Time$) for undistorted training and test data [12].

As can be seen, HTK obtained better results, since it applies Baum-Welch training, that considers all paths. In contrast, Kaldi is about two times faster, due to the fact that the Viterbi training only trains the best path to save time.

5.2 Different Pruning Beam-Widths

The idea of applying a pruning beam-width during training is to consider the N most probable paths and to train only the most probable path to reduce the computational complexity while not affecting the recognition rate. As already mentioned in Section 4.1, the pruning beam-width can be determined as input option to the monophone training. Table 5.2 shows the recognition results and computational times after monophone training using different pruning beam-widths.

	6	10	14	18	22	26	30
WER_{mono}	25.04	24.30	24.44	24.07	24.14	24.15	23.83
$Time$	30 min	30 min	31 min	33 min	35 min	38 min	41 min

Table 5.2: Comparison of WER_{mono} and computation time $Time$ for monophone training with different pruning beam-widths.

Since the values of the WER improve slightly and the computational time increases, setting the pruning beam-width is a compromise between recognition accuracy and computational time.

5.3 Reverberated Test Data

This section aims to depict the effect of reverberation on the recognition accuracy. The test data was reverberated as described in Section 4.2. WER_{mono} denotes the WER, that was obtained decoding after the monophone training. WER_{tri} describes the WER for training with the training data set train_si84, a triphone context-dependency model and using a trigram language model. As can be seen in Table 5.3, the WER increases significantly already for moderate reverberant acoustic environments like room A.

	clean	A	B	C	D	E
T_{60}	/	300 ms	600 ms	700 ms	780 ms	900 ms
WER_{mono}	25.34	69.70	86.50	88.59	90.84	93.23
WER_{tri}	12.37	54.65	84.30	84.88	88.71	93.02

Table 5.3: Comparison of WERs for reverberated test data in different acoustic environments: T_{60} is the reverberation time of the room, WER_{mono} is the WER for decoding without context-dependency, WER_{tri} is the WER for decoding a triphone context-dependency model with trigram language model.

5.4 Feature-space Transformation

This section presents recognition results, that were gained applying different feature-space transformations. We focused on two approaches: A combination of LDA and MLLT and a combination of SAT and fMLLR. All results were generated with the training data set train_si84, a triphone context-dependency model and using a trigram language model. The total number of Gaussian distributions was chosen to 15,000. The resulting WER is represented by WER_{tri} . For $WER_{LDA+MLLT}$, the Kaldi script steps/train_lda_mllt.sh was applied before decoding. The $WER_{SAT+fMLLR}$ results were obtained training the features with steps/train_sat.sh on top of the LDA+MLLT-features and decoding with the function steps/decode_fmllr.sh. The best results, denoted by $WER_{fMLLR-feats}$, were obtained by applying steps/make_fmllr_feats.sh, creating new alignments with steps/align_fmllr.sh, applying SAT and using the steps/decode_fmllr.sh

	clean	A	B	C	D	E
T_{60}	/	300 ms	600 ms	700 ms	780 ms	900 ms
WER_{tri}	12.37	54.65	84.30	84.88	88.71	93.02
$WER_{LDA+MLLT}$	9.39	61.35	87.36	88.02	90.41	92.54
$WER_{SAT+fMLLR}$	9.23	35.67	71.88	75.23	83.41	91.58
$WER_{fMLLR-feats}$	9.52	35.07	71.81	74.53	83.29	90.87

Table 5.4: Comparison of WERs for reverberated test data in different acoustic environments: T_{60} is the reverberation time of the room, WER_{tri} is the WER for decoding a triphone context-dependency model with trigram language model, $WER_{LDA+MLLT}$ is the WER using LDA+MLLT-features, $WER_{SAT+fMLLR}$ is the WER applying SAT and decode_fmllr.sh, $WER_{fMLLR-feats}$ is the WER obtained by generating fMLLR-features and using decode_fmllr.sh.

script for decoding. Table 5.4 summarizes these results.

To emphasize the achieved improvements, Figure 5.1 compares the WER_{tri} for decoding a triphone context-dependency model with trigram language model to $WER_{fMLLR-feats}$ applying fMLLR-features (see Table 5.4).

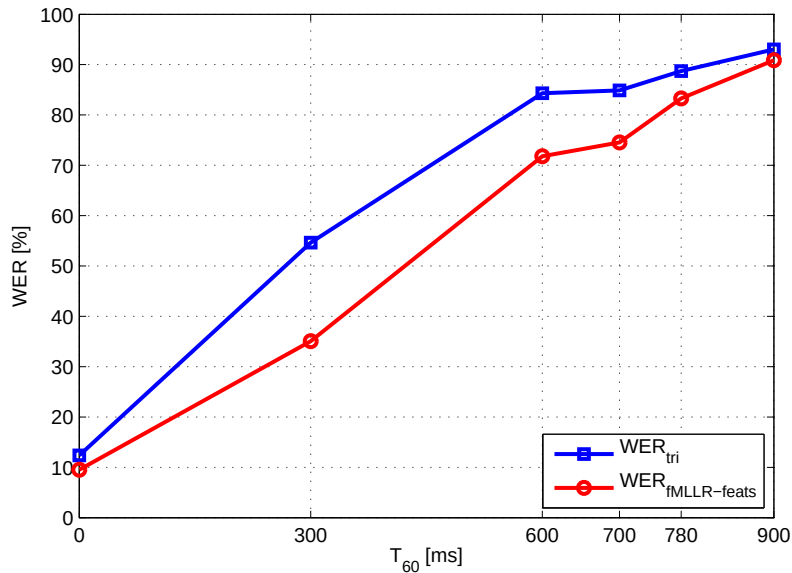


Figure 5.1: WER as a function of the reverberation time T_{60} .

6 Conclusions

This thesis presents the open-source Kaldi toolkit as an alternative to the HTK Toolkit for a LVCSR task. In Chapter 2 we illustrate, that the Kaldi Toolkit supports common stochastic approaches as HMMs and offers a high potential for speech recognition research. Among this, the linear feature-space transformations LDA+MLLT and SAT+fMLLR are presented. Furthermore, we describe the challenges in distant-talking speech recognition due to multi-path propagation of the sound waves in the acoustic environment. Chapter 3 pays attention to the implementation of HMMs in a speech recognition system by applying WFSTs. Via embedding the OpenFst library in Kaldi, WFSTs enable an efficient decoding graph creation according to the HCLG recipe. Following this, the speech recognition pipeline is explained in detail in Chapter 4. Every step is examined, including descriptions about crucial files and folders that are produced. Additionally, we describe the creation of reverberated test data which is subsequently used in Chapter 5. The experimental results are shown in Chapter 5: In order to evaluate the recognition accuracy in terms of the WER, the Kaldi Toolkit is compared to the HTK Toolkit, where we could show that the Kaldi Toolkit reaches an almost similar recognition accuracy at a much smaller computational time. Further, we examine the impact of the pruning beam-width on the recognition accuracy and the computational time. The results showed, that setting the pruning beam-width is a compromise between recognition accuracy and computational time. Finally, as the ultimate goal is to achieve robust results in distant-talking speech recognition, we investigated the performance of the speech recognition system (trained on clean-speech data) for reverberated test data in different acoustic environments. As viewed in Section 5.3, the

recognition rates decreased significantly, already for moderate acoustic environments ($T_{60} = 300$ ms). Subsequently, the feature-space transformations LDA+MLLT and SAT+fMLLR were applied, to investigate the improvements, that can be achieved with these methods. An average improvement of the WER of 8.81 % has been identified by using a linear feature-space transformation based on fMLLR.

Future research with the Kaldi Toolkit could include additional feature-space transformations and discriminative training methods to improve the recognition accuracy of the speech recognition system (based on HMMs and GMMs). This implicitly leads to an improved state-frame alignment which is the basis for training deep neural networks as state-of-the-art acoustic model (since 2011 [31]).

List of Figures

2.1	Illustration of the extraction of MFCC features [10].	5
2.2	Typical HMM topology used in ASR [10].	8
2.3	Illustration of the Viterbi training procedure [12].	10
2.4	Distant-talking ASR scenario [4].	12
2.5	Example of a RIR $h(n)$	13
2.6	MFCC representation of the utterance "four, two, seven": a) FVS of a clean speech signal, b) FVS of a reverberated speech signal [10].	14
3.1	Examples of WFSTs representing a) a language model and b) a lexicon [26].	17
3.2	Example of the composition of two transducers a) and b) [26].	18
3.3	Example of the determinization of WFSTs: a) a nondeterministic trans- ducer and b) an equivalent deterministic transducer [26].	20
3.4	Minimization of WFSTs: a) a deterministic transducer and b) an equiv- alent minimal transducer [26].	20
4.1	Overview of the generated files and folders during data preparation. . .	23
4.2	Overview of the generated files and folders during lexicon preparation. .	25
4.3	Overview of the generated files and folders during lexicon preparation. .	26
4.4	Overview of the generated files and folders during feature extraction. .	28
4.5	Overview of the generated files and folders during monophone training.	30
5.1	WER as a function of the reverberation time T_{60}	38

List of Tables

4.1	RIR characteristics of five rooms: T_{60} is the reverberation time, d is the distance between the acoustic source and the microphone, N is the number of RIRs measured for the room [12].	33
5.1	Comparison between Kaldi and HTK in terms of performance (WER) and computation time ($Time$) for undistorted training and test data [12].	36
5.2	Comparison of WER_{mono} and computation time $Time$ for monophone training with different pruning beam-widths.	36
5.3	Comparison of WERs for reverberated test data in different acoustic environments: T_{60} is the reverberation time of the room, WER_{mono} is the WER for decoding without context-dependency, WER_{tri} is the WER for decoding a triphone context-dependency model with trigram language model.	37
5.4	Comparison of WERs for reverberated test data in different acoustic environments: T_{60} is the reverberation time of the room, WER_{tri} is the WER for decoding a triphone context-dependency model with tri-gram language model, $WER_{LDA+MLLT}$ is the WER using LDA+MLLT-features, $WER_{SAT+fMLLR}$ is the WER applying SAT and decode_fmllr.sh, $WER_{fMLLR-feats}$ is the WER obtained by generating fMLLR-features and using decode_fmllr.sh.	38

Bibliography

- [1] T. Yoshioka, A. Sehr, M. Delcroix, K. Kinoshita, R. Maas, T. Nakatani, and W. Kellermann, “Making Machines Understand Us in Reverberant Rooms,” *Signal Processing Magazine, IEEE*, vol. 29, no. 6, pp. 114–126, 2012.
- [2] M. A. Hearst, “‘Natural’ Search User Interfaces,” *Communications of the ACM*, vol. 54, pp. 60–67, 2011.
- [3] M. A. Grasso, “Automated Speech Recognition in Medical Applications,” 1995.
- [4] A. Sehr, “Robust Speech Recognition Techniques for Changing Reverberation Conditions,” Master’s thesis, Chair of Multimedia Communications and Signal Processing, Friedrich-Alexander-Universität Erlangen-Nürnberg, Aug. 2010.
- [5] S. Matsoukas, R. Schwartz, H. Jin, and L. Nguyen, “Practical Implementations of Speaker-Adaptive Training,” in *DARPA Speech Recognition Workshop*, 1997.
- [6] D. B. Paul and J. M. Baker, “The Design for Wall Street Journal-based CSR Corpus,” in *DARPA Speech and Language Workshop*. Morgan Kaufmann Publishers, 1992, pp. 357–362.
- [7] J. G. et. al. (1994) November 1992 ARPA Continious Speech Recognition Benchmark Tests Corpora and Instructions. [Online]. Available: catalog.ldc.upenn.edu/docs/LDC93S6A/csrmov92.txt
- [8] M. J. F. Gales, “Model-Based Techniques for Noise Robust Speech Recognition,”

- Ph.D. dissertation, Gonville and Caius College, University of Cambridge, Sept. 1995.
- [9] C. Hofmann, “Evaluation of a Novel HMM Training Concept for Reverberation-Robust ASR,” Chair of Multimedia Communications and Signal Processing, Friedrich-Alexander-Universität Erlangen-Nürnberg, Project Work, Apr. 2010.
- [10] A. Sehr, “Reverberant Modeling for Robust Distant-Talking Speech Recognition,” Ph.D. dissertation, Chair of Multimedia Communications and Signal Processing, Friedrich-Alexander-Universität Erlangen-Nürnberg, Dec. 2010.
- [11] S. Young, D. Kershaw, J. Odell, D. Ollason, V. Valtchev, and P. Woodland, *The HTK Book Version 3.4*. Cambridge University Press, 2006.
- [12] H. Barfuß, “Extension of a Novel Hidden Markov Model Training Concept for Reverberation-Robust Speech Recognition to Large-Vocabulary Tasks,” Nov. 2012.
- [13] S. Sharma, D. Ellis, S. Kajarekar, P. Jain, and H. Hermansky, “Feature Extraction Using Non-Linear Transformation For Robust Speech Recognition On The Aurora Database,” in *Acoustics, Speech, and Signal Processing, 2000. ICASSP’00. Proceedings. 2000 IEEE International Conference on*.
- [14] G. Saon, M. Padmanabhan, R. Gopinath, and S. Chen, “Maximum Likelihood Discriminant Feature Spaces,” in *Acoustics, Speech, and Signal Processing, 2000. ICASSP ’00. Proceedings. 2000 IEEE International Conference on*, vol. 2. IEEE, 2000, pp. 1129–1132.
- [15] T. Anastasakos, J. McDonough, R. Schwartz, and J. Makhoul, “A Compact Model for Speaker-Adaptive Training,” in *Spoken Language, 1996. ICSLP 96. Proceedings., Fourth International Conference on*, vol. 2, Oct 1996, pp. 1137–1140.

- [16] Z. Zajíc, L. Machlica, and L. Müller, “Refinement Approach for Adaptation Based on Combination of MAP and fMLLR,” *Lecture Notes in Computer Science*, vol. 5729/2009, pp. 274–281, 2009.
- [17] M. J. F. Gales, “Maximum Likelihood Linear Transformations for HMM-Based Speech Recognition,” *Computer speech & language*, vol. 12, no. 2, pp. 75–98, 1998.
- [18] D. Povey, H.-K. J. Kuo, and H. Soltau, “Fast Speaker Adaptive Training for Speech Recognition,” in *INTERSPEECH*, 2008, pp. 1245–1248.
- [19] J. Vanek and Z. Zajíc, “A Direct Criterion Minimization Based fMLLR via Gradient Descend,” in *Text, Speech, and Dialogue*, pp. 52–59.
- [20] E. G. Schukat-Talamazzini, *Automatische Spracherkennung: Statistische Verfahren der Musteranalyse*. Vieweg Verlag, 1995.
- [21] X. Huang, A. Acero, and H. W. Hon, *Spoken Language Processing: A Guide to Theory, Algorithm, and System Development*. Prentice Hall, Inc., Upper Saddle River, NJ, USA, 2011.
- [22] W. Liu and H. Weisheng, “Improved Viterbi Algorithm in Continuous Speech Recognition,” in *Computer Application and System Modeling (ICCASM), 2010 International Conference on*, vol. 7, Oct 2010, pp. 207–209.
- [23] [Online]. Available: kaldi.sourceforge.net/about.html
- [24] D. P. et al., “The Kaldi Speech Recognition Toolkit,” in *IEEE 2011 Workshop on Automatic Speech Recognition and Understanding*. IEEE Signal Processing Society, Dec. 2011, pp. 1–4.
- [25] [Online]. Available: kaldi.sourceforge.net/dnn.html
- [26] M. Mohri, F. Pereira, and M. Riley, “Speech Recognition with Weighted Finite-State Transducers,” in *Springer Handbook of Speech Processing*. Springer Berlin Heidelberg, 2008, pp. 559–584.

- [27] [Online]. Available: kaldi.sourceforge.net/graph.html
- [28] [Online]. Available: kaldi.sourceforge.net/tutorial_running.html
- [29] [Online]. Available: kaldi.sourceforge.net/feat.html
- [30] D. Klakow and J. Peters, “Testing the correlation of word error rate and perplexity,” *Speech Communication*, vol. 38, no. 1, pp. 19–28, 2002.
- [31] G. H. et al., “Deep Neural Networks for Acoustic Modeling in Speech Recognition: The shared views of four research groups,” *Signal Processing Magazine, IEEE*, vol. 29, no. 6, pp. 82–97, 2012.